

Unidad 4: Agilidad en el Desarrollo de Software

Semana 2: Kanban, Lean y la Excelencia Técnica con XP

Introducción: Más allá de los Sprints

Si bien Scrum es el marco ágil más popular y estructurado, su enfoque en iteraciones fijas de tiempo (Sprints) no es universalmente aplicable. Existen contextos de trabajo, como el mantenimiento de *software*, el soporte técnico, la gestión de infraestructura (DevOps) o los equipos de I+D (Investigación y Desarrollo), donde la llegada de tareas es **imprevisible y prioritaria**, haciendo que la rigidez del *Sprint* sea contraproducente.

Esta semana, el foco se desplaza hacia la **gestión del flujo continuo** con **Kanban**, la **optimización de procesos** mediante la filosofía **Lean** y, fundamentalmente, la **elevación de la calidad del código** a través de las prácticas rigurosas de **Extreme Programming (XP)**. La visión moderna (2026) del profesional ágil es la de un **Arquitecto de Metodologías**, capaz de aplicar un enfoque "híbrido" o *tailoring* según la cultura de la empresa, el tipo de proyecto y las necesidades específicas del equipo, integrando lo mejor de cada marco.

1) Kanban: La gestión visual del flujo

El concepto de **Kanban**, que significa "tarjeta visual" o "señal" en japonés, nació en los años 40 y 50 en las líneas de producción de **Toyota** bajo la dirección de Taiichi Ohno. Su objetivo original era alinear el inventario con la demanda del cliente. David J. Anderson lo adaptó a inicios de los 2000 para el desarrollo de *software* y el trabajo de conocimiento, centrándose en el flujo de valor.

A diferencia de Scrum, que es un marco prescriptivo que define roles, eventos y artefactos, Kanban es un **método de cambio evolutivo**. No requiere *Sprints* ni roles obligatorios como el *Scrum Master*, sino que se centra en optimizar el camino que recorre una tarea desde la idea hasta el cliente. Su éxito radica en el enfoque en **entrega continua** (*flow*): no se trata de "cerrar un Sprint", sino de **mover trabajo hacia la entrega de forma constante y predecible**.

1.1 Los 4 principios fundamentales de Kanban (pilares del cambio)

Kanban se presenta como una capa de mejora continua que se superpone al proceso actual, lo que facilita su adopción sin traumas organizacionales:

1. **Empezar con lo que haces ahora (Start with what you do now):** Es el principio más poderoso. No se exige dismantelar la estructura actual del equipo. Kanban se implementa sobre el proceso de

trabajo existente (el tablero puede reflejar las fases actuales: Análisis, Desarrollo, QA, Despliegue) para exponer sus ineficiencias y, a partir de ahí, mejorarlo gradualmente.

2. **Comprometerse a buscar el cambio evolutivo (Agree to pursue incremental, evolutionary change):** El lema es *Kaizen* (mejora continua y pequeña). Se rechazan los cambios drásticos (*Big Bang Change*). Las mejoras se proponen, se implementan y se miden con el objetivo de reducir el tiempo que tarda una tarea en completarse.
 3. **Respetar los roles, responsabilidades y títulos actuales (Respect current roles, responsibilities, and job titles):** Kanban valora la experiencia y estructura ya establecidas. La optimización se enfoca en *cómo* el trabajo fluye entre esas funciones, no en *quién* hace qué, minimizando la resistencia al cambio.
 4. **Fomentar actos de liderazgo en todos los niveles (Encourage acts of leadership at all levels):** La mejora del flujo es responsabilidad de todos. Desde el programador *junior* que nota un *cuello de botella* en el *testing* hasta el gerente que analiza las métricas, cualquiera puede y debe sugerir una modificación en la configuración del tablero o de las políticas.
-

1.2 Las 6 prácticas esenciales (herramientas de control)

1. **Visualizar el trabajo:** El tablero es la verdad. Debe ser transparente para todo el equipo y mostrar cada paso del proceso (columnas) y cada tarea (*tarjeta*). La máxima es: **Si no está en el tablero, no existe**. Esto permite identificar inmediatamente los *cuellos de botella* y el estado real del trabajo.
2. **Limitar el Trabajo en Curso (WIP - Work In Progress):** Esta es la regla de oro y el corazón de Kanban. El *WIP Limit* es un tope máximo de tarjetas permitidas en una columna o en todo el sistema. Su objetivo no es acelerar el trabajo, sino **forzar la finalización de las tareas existentes antes de empezar nuevas**. Al limitar el WIP, se genera un *efecto pull* (tirar): cuando un desarrollador termina una tarea, va a la columna anterior y "jala" la siguiente tarea, en lugar de que el sistema le "empuje" nuevas tareas constantemente, eliminando así el *context switching* y acelerando la entrega.
3. **Gestionar el flujo:** El foco está en la velocidad y la predictibilidad. El objetivo no es mantener a la gente ocupada, sino que las tareas se muevan de izquierda a derecha (el flujo de valor) de la forma más suave, rápida y predecible posible. El *Cycle Time* (tiempo de ciclo) es la métrica principal para medir la salud del flujo.
4. **Hacer las políticas explícitas (Make Process Policies Explicit):** Las reglas de movimiento de las tarjetas deben ser conocidas por todos. Por ejemplo, la **Definition of Done (DoD)** de la columna "Desarrollo" podría ser: "Código revisado, cobertura de pruebas del 90%, sin errores críticos conocidos." Si una tarjeta no cumple la política, no puede pasar a la siguiente columna (ej., "Listo

para Testing").

5. **Implementar bucles de retroalimentación (Implement Feedback Loops):** Se mantienen reuniones cortas y enfocadas para revisar el flujo:
 - **Kanban Meeting (Daily):** Similar al *Daily Scrum*, pero enfocado en el **flujo**. Se recorre el tablero de derecha a izquierda (*pull*), preguntando: "¿Qué debemos terminar hoy?"
 - **Service Delivery Review:** Revisión con el cliente sobre el desempeño y la predictibilidad del servicio.
 - **Operations Review:** Revisión del sistema Kanban en sí mismo (similar a la retrospectiva), usando las métricas como base para la mejora.
 6. **Mejorar colaborativamente y evolucionar experimentalmente (Improve Collaboratively, Evolve Experimentally):** Las métricas (como el *Lead Time* y el *Throughput*) se utilizan como evidencia objetiva para proponer e implementar mejoras (hipótesis). Si una mejora no funciona, se revierte. Es un ciclo de mejora basado en el método científico.
-

1.3 Kanban avanzado: optimización con Clases de Servicio

Kanban utiliza las **Clases de Servicio** para gestionar la **prioridad** y la **expectativa de entrega** de diferentes tipos de tareas, manteniendo el flujo continuo:

- **Estándar:** Tareas regulares, se mueven con la velocidad normal del sistema.
 - **Fecha Fija (Fixed Date):** Tareas que deben entregarse en una fecha específica (ej., un requisito regulatorio). Tienen prioridad sobre las estándar a medida que se acerca la fecha.
 - **Urgente (Expedite):** Tareas críticas que detienen el negocio (ej., un error grave en producción). Rompen temporalmente el límite WIP y se mueven inmediatamente a través del tablero.
 - **Intangible:** Tareas grandes o proyectos de I+D que no tienen un plazo inmediato. Se trabajan en segundo plano o se espera a que el *Lead Time* promedio sea bajo para empezarlas.
-

2) Lean Software Development: eliminar el desperdicio

La metodología **Lean** es una filosofía de gestión que se centra en una pregunta clave: **¿Qué es valor para el cliente?** Su objetivo es maximizar el valor entregado minimizando al mismo tiempo todo aquello que se considera **desperdicio** (*Muda* en japonés). En el desarrollo de *software*, el desperdicio es cualquier actividad, código, proceso o espera que consume tiempo o recursos sin añadir valor al producto final desde la perspectiva del cliente.

Los principios de Lean en *software* (formalizados por Mary y Tom Poppendieck) se basan en la eliminación de estos desperdicios y la entrega rápida

2.1 Los 7 desperdicios del software (análisis detallado)

- **Trabajo hecho parcialmente (Partially Done Work):**
 - **Descripción:** Es el código que se ha escrito, pero que no ha sido probado, integrado o desplegado al cliente. En términos económicos, es dinero invertido que aún no ha generado retorno.
 - **Ejemplo:** Ramas de Git con código a medio terminar que nunca se fusionan. Tareas en la columna "En desarrollo" que permanecen allí por semanas.
 - **Solución Ágil:** Limitar el WIP (Kanban) y priorizar la **Integración Continua** (XP).
- **Procesos extra (Extra Processing):**
 - **Descripción:** Burocracia excesiva, reuniones innecesarias con demasiados asistentes, múltiples firmas de aprobación para decisiones menores, o documentación detallada de funcionalidades que están en constante cambio.
 - **Ejemplo:** Una reunión diaria que dura 45 minutos en lugar de 15, o un documento de requisitos de 100 páginas para un MVP (Producto Mínimo Viable).
 - **Solución Ágil:** Fomentar la **Comunicación cara a cara** (XP) y hacer las **políticas explícitas** (Kanban) para solo documentar lo necesario.
- **Características extra (Gold Plating):**
 - **Descripción:** El equipo desarrolla funcionalidades o añade tecnología que el cliente no ha solicitado, que no usará o que no añade valor. Es la tendencia del desarrollador a "sobre-ingenierar" la solución.
 - **Ejemplo:** Pasar tres días implementando un sistema de *cacheo* avanzado cuando la aplicación apenas tiene 100 usuarios.
 - **Solución Ágil:** **Simplicidad** (XP), preguntando "¿Cuál es lo más sencillo que funciona?" y basar el desarrollo en la **retroalimentación constante del cliente**.
- **Cambio de tareas (Context Switching):**
 - **Descripción:** El costo cognitivo y temporal de saltar de un proyecto o tarea a otra. Cada vez que una persona interrumpe su trabajo, se pierde tiempo de concentración para retomar la tarea anterior.
 - **Ejemplo:** Un desarrollador que trabaja 3 horas en la Tarea A, luego cambia 2 horas a la Tarea B por una interrupción, y después intenta volver a la Tarea A.
 - **Solución Ágil:** La limitación del WIP es el método directo para combatir este desperdicio, obligando al foco.
- **Esperas (Waiting):**
 - **Descripción:** Cualquier momento en que un miembro del equipo no puede avanzar porque

depende de otro recurso o persona.

- **Ejemplo:** Programadores esperando que el equipo de *DevOps* configure un entorno, *testers* esperando que el código se despliegue, o el *Product Owner* esperando la aprobación de un diseño.
 - **Solución Ágil: Gestionar el flujo** (Kanban) para identificar y eliminar los *cuellos de botella* que causan las esperas.
- **Movimientos y Handoffs (Handoffs):**
 - **Descripción:** La transferencia de trabajo e información de una persona/equipo a otra. En cada *handoff* se pierde contexto, se crean malentendidos y se introducen defectos.
 - **Ejemplo:** Un documento de especificación pasa del Analista al Diseñador, de este al Programador, y luego al Tester. Cada paso es un potencial error de interpretación.
 - **Solución Ágil: Programación en Pareja** (XP) y equipos **multifuncionales** para reducir las transferencias.
 - **Defectos (Defects):**
 - **Descripción:** Errores o *bugs* que requieren trabajo doble (el tiempo de corregir el error más el tiempo que se invirtió originalmente en crearlo).
 - **Ejemplo:** Un error reportado en producción una semana después de un despliegue.
 - **Solución Ágil: TDD** (Desarrollo Dirigido por Pruebas) y **Programación en Pareja** (XP) para encontrar y prevenir defectos lo antes posible.
-

2.2 💡 Visión 2026: IA como reductor de desperdicios

La Inteligencia Artificial se está integrando directamente en la cadena de valor ágil para atacar desperdicios específicos:

- **Generación de código repetitivo:** Herramientas como GitHub Copilot o Gemini generan código *boilerplate* y tests unitarios, reduciendo el *Trabajo hecho parcialmente* y los *Defectos*.
 - **Documentación base:** *Gems* o *plugins* en Docs y Gmail pueden generar resúmenes de reuniones o documentación técnica a partir del código fuente, eliminando el desperdicio de *Procesos extra*.
-

3) 🛠️ Extreme Programming (XP): la ingeniería de excelencia

Si Scrum provee la estructura de gestión y Kanban la optimización del flujo, **Extreme Programming (XP)** provee el **motor técnico** para asegurar que el producto entregado sea de la más alta calidad y que el ritmo de desarrollo sea sostenible. XP es una metodología centrada en el bienestar del desarrollador y en la producción de *software* robusto, adaptable y de alto valor.

3.1 Valores de XP

- **Comunicación:** El valor más importante. Se promueve el diálogo constante (código claro, reuniones mínimas, *Pair Programming*).
- **Simplicidad:** "Hacer lo más sencillo que funcione hoy." Se rechaza la *Gold Plating* y la sobre-ingeniería.
- **Retroalimentación:** Obtener *feedback* rápido en todos los niveles: del código (pruebas automáticas), del sistema (Integración Continua) y del cliente (entregas frecuentes).
- **Coraje:** La valentía para tomar decisiones difíciles: **Atreverse a borrar código malo y reescribirlo** (Refactorización), y la valentía de decir "no" a las características innecesarias.

3.2 Prácticas técnicas clave (profundización)

- **Programación en Pareja (Pair Programming):**
 - **Concepto:** Dos desarrolladores, una sola computadora, un solo teclado. Uno es el **Conductor** (*Driver*), que escribe el código; el otro es el **Navegante** (*Navigator*), que revisa la lógica, piensa en el panorama general, y sugiere las pruebas a escribir. Los roles se intercambian frecuentemente (cada 15-30 minutos).
 - **Beneficio Principal:**
 - **Reducción de defectos:** Dos ojos ven más errores que uno.
 - **Transferencia de conocimiento (bus factor):** Si un desarrollador se va, el otro conoce el código.
 - **Mejor diseño:** El diálogo constante obliga a un diseño más simple y limpio.
- **Desarrollo Dirigido por Pruebas (TDD - Test Driven Development):**
 - **Concepto:** Es una disciplina de desarrollo basada en un ciclo de 3 pasos (Rojo-Verde-Refactorizar).
 - **ROJO:** Escribir primero una **prueba automatizada** que falle. Esto define exactamente lo que se espera de la funcionalidad.
 - **VERDE:** Escribir la **cantidad mínima de código** necesaria para que esa prueba pase.
 - **REFACTORIZAR:** Mejorar el código que acaba de funcionar (limpieza, renombrar variables, simplificar) sin que las pruebas dejen de pasar.
 - **Beneficio Principal:**
 - **Diseño mejorado:** TDD obliga a escribir código que es *testable*, lo que por naturaleza es más modular y desacoplado.
 - **Garantía de calidad:** Cada funcionalidad tiene su *red de seguridad* de pruebas automáticas.
- **Integración Continua (CI - Continuous Integration):**
 - **Concepto:** Subir el código al repositorio principal (**rama master** o **rama main**) de forma

frecuente, idealmente varias veces al día. Cada subida dispara un proceso automatizado que compila, ejecuta todas las pruebas unitarias y de integración, y verifica que el *build* sea exitoso.

- **Práctica Aseguradora de la Calidad: CI asegura que el código siempre esté en un estado integrable y funcional.** Si una integración rompe las pruebas, debe revertirse o corregirse inmediatamente. Reduce la complejidad de las fusiones tardías.
- **Refactorización (Refactoring):**
 - **Concepto:** La disciplina de **mejorar la estructura interna del código** (limpieza, legibilidad, modularidad, rendimiento) **sin cambiar su comportamiento externo o la funcionalidad** para el usuario. Es el antídoto contra la **Deuda Técnica**.
 - **Beneficio Principal:** Mantiene la **simplicidad** y el **coraje** vivo. Un código limpio es más fácil de mantener, de extender y reduce el costo de añadir nuevas funcionalidades. Debe ser una práctica diaria, no un proyecto de semanas.

4) Métricas ágiles: ¿cómo sabemos si somos eficientes?

La Agilidad se basa en la transparencia y en el uso de datos para tomar decisiones objetivas (empirismo). Las métricas ágiles no miden el esfuerzo (*horas trabajadas*) sino el **resultado** y la **eficiencia del flujo**.

Métrica	Definición	Relevancia
Lead Time	Tiempo total desde que una idea o solicitud entra en el sistema (ej., aparece en el <i>Backlog</i>) hasta que está terminada y en manos del cliente.	Mide la efectividad total del proceso de principio a fin. Es lo que realmente le importa al cliente.
Cycle Time	El tiempo que transcurre desde que el equipo empieza a trabajar en una tarea (ej., pasa a la columna "Haciendo") hasta que está terminada y lista para desplegar.	Mide la eficiencia operativa interna del equipo de desarrollo. Se usa para predecir el futuro (<i>predictibilidad</i>).
Throughput (Rendimiento)	Cuántas tareas (tarjetas, historias de usuario) el equipo termina y entrega en un periodo de tiempo (ej., por semana o por mes).	Mide la capacidad de entrega del equipo. Un <i>Throughput</i> estable y creciente indica un flujo saludable.
Velocity (Velocidad)	Específica de Scrum. Mide la suma de los "puntos de historia" que el equipo completa por Sprint .	Se usa para la planificación de Sprints . Permite al equipo predecir cuántas tareas pueden comprometerse a hacer en la siguiente iteración.

5) Enfoques híbridos: Scrumban y la realidad laboral

En la práctica, muchas empresas no siguen una sola metodología: aplican modelos **híbridos** o **Glocal** (*globalmente pensado, localmente adaptado*).

Scrumban: combinación de disciplina y flujo

Scrumban surge de la necesidad de usar disciplina temporal de Scrum en entornos que se benefician del flujo continuo de Kanban (por ejemplo, equipos de mantenimiento que además desarrollan nuevas funcionalidades).

Componentes de Scrumban

Componente	Origen	Beneficio
Estructura de Reuniones y Roles	Scrum	Mantiene la disciplina y cadencia. Se usan <i>Dailies</i> , <i>Planning</i> y <i>Retrospectivas</i> . Los roles de <i>Product Owner</i> y <i>Scrum Master</i> suelen estar presentes.
Flujo Continuo y WIP Limit	Kanban	Permite que las tareas críticas de mantenimiento o soporte rompan el <i>Sprint</i> sin destruirlo, asegurando que el flujo de valor nunca se detenga.
Métricas	Ambos	Puede usar <i>Velocity</i> para el <i>Planning</i> de las tareas recurrentes y <i>Cycle Time</i> para la predictibilidad de las tareas críticas.

Es el modelo más recomendado para equipos que están comenzando su carrera profesional, porque combina estructura (fácil de aprender) y flexibilidad (manejo de imprevistos).

6) Herramientas líderes en la industria (actualidad 2026)

Un profesional ágil debe dominar las herramientas que digitalizan estos marcos de trabajo.

1. **Jira Software:** El estándar corporativo. Está diseñado para gestionar *Backlogs* y *Sprints* de Scrum, pero sus capacidades de tablero y flujos de trabajo (*workflows*) lo hacen igualmente potente para implementar Kanban y Scrumban. Su fortaleza radica en la trazabilidad, los informes detallados de *Velocity* y *Lead Time*, y su capacidad de integrarse con casi cualquier herramienta de desarrollo (Git, CI/CD).
2. **Azure DevOps:** Muy común en entornos que usan tecnologías de Microsoft (.NET, Azure Cloud). Es una suite completa que ofrece gestión de proyectos (tableros Kanban/Scrum), repositorios de

código (Git), *pipelines* de CI/CD y gestión de pruebas, todo integrado bajo una misma plataforma.

3. **Linear:** Una herramienta moderna, minimalista y muy rápida, que está ganando terreno en *startups* de IA y tecnología. Su foco es la eficiencia del desarrollador, con una fuerte implementación del flujo Kanban y atajos de teclado rápidos. Se especializa en reducir la burocracia para enfocarse en la entrega.
4. **Trello:** Ideal para Kanban simple y proyectos pequeños. Es altamente visual y fácil de usar. Aunque carece de la robustez métrica de Jira, es la herramienta perfecta para la **Actividad de la Semana** donde se aplica un tablero Kanban a la vida universitaria.

Ecosistema tecnológico moderno: más allá de Trello

Aunque Trello es excelente para introducir Kanban, equipos de alto rendimiento (2025–2026) adoptan plataformas con mayor profundidad:

- **Linear:** pensada para ingeniería; interfaz ultra rápida, atajos y automatizaciones integradas con repositorios.
- **Notion:** “cerebro” del proyecto; integra Wiki/documentación, tareas y gestión del conocimiento en un espacio colaborativo.



Actividad de la Semana 2

"El Tablero de tu Vida Universitaria"

1. Crea un tablero Kanban (en papel o digital como Trello) para gestionar tus materias del semestre.
2. Define tus columnas: *Por hacer*, *En lectura*, *Haciendo Trabajo Práctico*, *Listo para rendir*, *Aprobada*.
3. **Aplica un WIP Limit:** No puedes tener más de 2 materias en la columna "Haciendo Trabajo Práctico".
4. Reflexiona: ¿Qué pasa cuando intentas mover una tercera materia a esa columna? ¿Cómo te obliga esto a priorizar?



Cierre de la Unidad 4

Han completado el recorrido por las metodologías que mueven al mundo tecnológico. Ahora saben que ser ágil no es trabajar más horas, sino **trabajar con inteligencia**: visualizar el proceso, eliminar lo que no sirve y enfocarse siempre en la calidad técnica.

Con esta base, están listos para la siguiente unidad, donde veremos cómo aplicar estos conceptos al mundo de las **Startups** y la creación de empresas de base tecnológica.